

Week 3 - WebApi Handson 3

Folder Structure

```
WebApi_Handson3/  
|  
├── WebApi_Handson3.csproj  
├── Program.cs  
├── appsettings.json  
├── appsettings.Development.json  
├── README.md  
├── Controllers/  
|   └── EmployeeController.cs  
├── Models/  
|   ├── Employee.cs  
|   ├── Department.cs  
|   └── Skill.cs  
├── Filters/  
|   ├── CustomAuthFilter.cs  
|   └── CustomExceptionFilter.cs  
├── Properties/  
|   └── launchSettings.json  
└── .vscode/  
    ├── launch.json  
    ├── tasks.json  
    └── settings.json
```

Software Requirements

- Visual Studio Code / Visual Studio 2022
- .NET SDK 8.0
- ASP.NET Core Web API
- Swagger
- WebApiCompatShim Package

Step 1 - Create the Project

Open Terminal and execute:

```
dotnet new webapi -n WebApi_Handson3
```

```
cd WebApi_Handson3
```

Install the required package:

```
dotnet add package Microsoft.AspNetCore.Mvc.WebApiCompatShim
```

File 1 – WebApi_Handson3.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

<PropertyGroup>

<TargetFramework>net8.0</TargetFramework>

<Nullable>enable</Nullable>

<ImplicitUsings>enable</ImplicitUsings>

</PropertyGroup>

<ItemGroup>

<PackageReference Include="Microsoft.AspNetCore.Mvc.WebApiCompatShim"
Version="2.3.0" />

<PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />

</ItemGroup>

</Project>
```

File 2 – Program.cs

```
using WebApi_Handson3.Filters;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers(options =>

{

    options.Filters.Add<CustomExceptionFilter>();

});
```

```
builder.Services.AddScoped<CustomAuthFilter>();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
var app = builder.Build();  
app.UseSwagger();  
app.UseSwaggerUI();  
app.UseHttpsRedirection();  
app.UseAuthorization();  
app.MapControllers();  
app.Run();
```

File 3 – appsettings.json

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "AllowedHosts": "*" }  
}
```

File 4 – Models/Department.cs

```
namespace WebApi_Handson3.Models
{
    public class Department
    {
        public int Id { get; set; }

        public string Name { get; set; } = string.Empty;
    }
}
```

File 5 – Models/Skill.cs

```
namespace WebApi_Handson3.Models
{
    public class Skill
    {
        public int Id { get; set; }

        public string Name { get; set; } = string.Empty;
    }
}
```

File 6 – Models/Employee.cs

```
using System;

using System.Collections.Generic;

namespace WebApi_Handson3.Models
{
    public class Employee
```

```

{
    public int Id { get; set; }

    public string Name { get; set; } = string.Empty;

    public int Salary { get; set; }

    public bool Permanent { get; set; }

    public Department Department { get; set; } = new Department();

    public List<Skill> Skills { get; set; } = new List<Skill>();

    public DateTime DateOfBirth { get; set; }
}
}

```

File 7 – Filters/CustomAuthFilter.cs

```

using Microsoft.AspNetCore.Mvc;

using Microsoft.AspNetCore.Mvc.Filters;

namespace WebApi_Handson3.Filters{

    public class CustomAuthFilter : ActionFilterAttribute {

        public override void OnActionExecuting(ActionExecutingContext context){

            if (!context.HttpContext.Request.Headers.ContainsKey("Authorization")){

                context.Result = new BadRequestObjectResult(

                    "Invalid request - No Auth token");

                return;

            }

            var token = context.HttpContext.Request.Headers["Authorization"].ToString();

            if (!token.Contains("Bearer"))

            {

                context.Result = new BadRequestObjectResult(

```

```

        "Invalid request - Token present but Bearer unavailable");
    }
}
}
}

```

File 8 – Filters/CustomExceptionHandler.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;
namespace WebApi_Handson3.Filters
{
    public class CustomExceptionHandler : IExceptionHandler
    {
        public void OnException(ExceptionContext context)
        {
            string path = "ExceptionLog.txt";
            File.AppendAllText(
                path,
                DateTime.Now +
                Environment.NewLine +
                context.Exception.ToString() +
                Environment.NewLine +
                "-----" +
                Environment.NewLine);
            context.Result = new ObjectResult(
                "Internal Server Error")

```

```

    {
        StatusCode = 500
    };
    context.ExceptionHandled = true;
}
}
}

```

File 9 – Controllers/EmployeeController.cs

```

using Microsoft.AspNetCore.Mvc;
using WebApi_Handson3.Filters;
using WebApi_Handson3.Models;
namespace WebApi_Handson3.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [ServiceFilter(typeof(CustomAuthFilter))]
    public class EmployeeController : ControllerBase
    {
        private static List<Employee> employees;

        public EmployeeController()
        {
            employees = GetStandardEmployeeList();
        }
    }
}

```

```

[HttpGet]

[ProducesResponseType(typeof(List<Employee>), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public ActionResult<List<Employee>> Get()
{
    return Ok(employees);
}

[HttpGet("{id}")]
public ActionResult<Employee> Get(int id)
{
    var emp = employees.FirstOrDefault(e => e.Id == id);

    if (emp == null)
        return NotFound();

    return Ok(emp);
}

[HttpPost]
public ActionResult<Employee> Post([FromBody] Employee employee)
{
    employees.Add(employee);

    return Ok(employee);
}

[HttpPut("{id}")]
public ActionResult<Employee> Put(int id, [FromBody] Employee employee)
{
    var emp = employees.FirstOrDefault(e => e.Id == id);

```



```

    if (emp == null)

        return BadRequest("Invalid Employee Id");

    emp.Name = employee.Name;

    emp.Salary = employee.Salary;

    emp.Permanent = employee.Permanent;

    emp.Department = employee.Department;

    emp.Skills = employee.Skills;

    emp.DateOfBirth = employee.DateOfBirth;

    return Ok(emp);
}

private static List<Employee> GetStandardEmployeeList()
{
    return new List<Employee>
    {
        new Employee
        {
            Id = 1,
            Name = "John",
            Salary = 50000,
            Permanent = true,
            DateOfBirth = new DateTime(1998,5,20),

            Department = new Department
            {
                Id = 1,

```

```

        Name = "IT"
    },
    Skills = new List<Skill>
    {
        new Skill{ Id=1, Name="C#" },
        new Skill{ Id=2, Name="ASP.NET Core" }
    }
},
new Employee
{
    Id = 2,
    Name = "David",
    Salary = 45000,
    Permanent = false,
    DateOfBirth = new DateTime(2000,10,12),
    Department = new Department
    {
        Id = 2,
        Name = "HR"
    },
    Skills = new List<Skill>
    {
        new Skill{ Id=3, Name="SQL" },
        new Skill{ Id=4, Name="Communication" }
    }
}

```

```
    };  
  }  
}  
}
```

File 10 – Properties/launchSettings.json

```
{  
  "$schema": "https://json.schemastore.org/launchsettings.json",  
  
  "profiles": {  
    "http": {  
      "commandName": "Project",  
      "launchBrowser": true,  
      "launchUrl": "swagger",  
      "applicationUrl": "http://localhost:5100",  
      "environmentVariables": {  
        "ASPNETCORE_ENVIRONMENT": "Development"  
      }  
    }  
  }  
}
```

Testing Using Swagger

1. Run the application:

```
dotnet restore
```

```
dotnet build
```

dotnet run

2. Open:

<http://localhost:5100/swagger>

3. Test the following endpoints:

- GET /api/Employee
- GET /api/Employee/{id}
- POST /api/Employee
- PUT /api/Employee/{id}

4. Include the Authorization header:

Authorization: Bearer sampletoken

Expected Output

- The GET API returns a list of employees.
 - The GET by ID API returns the requested employee.
 - The POST API adds a new employee.
 - The PUT API updates an existing employee.
 - Missing or invalid Authorization header returns a **400 Bad Request**.
 - Exceptions are logged to ExceptionLog.txt and return **500 Internal Server Error**.
-